

Doc-Doctor 测试文档

目录

- 1. [测试概述](#)
- 2. [测试环境配置](#)
- 3. [运行测试](#)
- 4. [测试覆盖范围](#)
- 5. [测试用例详细说明](#)
- 6. [测试数据](#)
- 7. [测试工具](#)

1. 测试概述

1.1 测试目标

Doc-Doctor 的测试旨在确保以下方面：

测试目标	说明
功能正确性	所有核心功能按预期工作
边界处理	正确处理各种边界情况和异常场景
集成稳定性	各模块之间能够正确协作
配置灵活性	白名单和配置功能正常工作
错误恢复	在异常情况下系统能够优雅降级

1.2 测试架构

```
src/test/
├── unit/                # 单元测试
│   ├── fileCheck.test.ts    # 文件解析和检查测试
│   ├── functionCheck.test.ts # 函数注释检查测试
│   ├── fileWhiteList.test.ts # 白名单功能测试
│   └── jumpToLocation.test.ts # 代码跳转功能测试
├── integration/          # 集成测试
│   ├── projectCheck.test.ts # 项目全量检查测试
│   └── database.test.ts     # 数据库操作测试
├── utils/                # 测试工具
│   └── testHelpers.ts      # 测试辅助函数
├── fixtures/             # 测试数据
│   └── testFiles.ts        # C/C++ 测试文件样本
├── extension.test.ts     # 扩展入口测试
└── README.md             # 快速参考文档
```

1.3 测试类型说明

测试类型	位置	目的	特点	覆盖范围
单元测试	src/test/unit/	测试单个模块的独立功能	快速执行、隔离依赖、使用 Mock 对象	文件解析、函数检查、白名单匹配、位置跳转等
集成测试	src/test/integration/	测试多个模块协同工作的场景	可能涉及文件系统、数据库、VS Code API	项目全量检查、数据库持久化、取消机制等
扩展测试	src/test/extension.test.ts	测试扩展的激活和命令注册	需要 VS Code 测试环境	扩展激活、命令注册

2. 测试环境配置

2.1 前置条件

软件/工具	版本要求	说明
Node.js	22.x 或更高	JavaScript 运行时环境
VS Code	1.105.0 或更高	代码编辑器
TypeScript	5.9.2 或更高	编程语言
npm	最新稳定版	包管理器，用于安装依赖

2.2 依赖安装

```
cd doc-doctor
npm install
```

2.3 编译代码

在运行测试前，需要先编译 TypeScript 代码：

```
npm run compile
```

2.4 测试配置

测试使用 `@vscode/test-electron` 框架，配置文件位于 `.vscode-test.mjs`。

配置项	值	说明
测试文件路径	<code>out/test/**/*.test.js</code>	编译后的 JavaScript 测试文件
测试运行器	VS Code Extension Test Runner	VS Code 扩展测试运行器

2.5 数据库测试说明

场景	行为	说明
C++ DLL 不可用	自动使用 Mock 模式	确保测试可在无原生模块环境中运行
Mock 模式	所有数据库操作返回成功	不实际持久化数据，仅用于测试验证

3. 运行测试

3.1 使用 VS Code 调试器

步骤 1：编译代码

在运行测试前，需要先编译 TypeScript 代码：

```
npm run compile
```

步骤 2：配置调试环境

在 `.vscode/launch.json` 中配置调试配置：

```
{
  "type": "extensionHost",
  "request": "launch",
  "name": "Extension Tests",
  "runtimeExecutable": "${execPath}",
  "args": [
    "--extensionDevelopmentPath=${workspaceFolder}",
    "--extensionTestsPath=${workspaceFolder}/out/test"
  ],
  "outFiles": ["${workspaceFolder}/out/**/*.js"],
  "preLaunchTask": "npm: compile"
}
```

步骤 3：运行调试

1. 在测试文件中设置断点（可选）
2. 打开调试视图（Activity Bar → Run and Debug，或使用快捷键 `Ctrl/Cmd + Shift + D`）
3. 选择 "Extension Tests" 配置
4. 按 F5 开始调试，或点击调试视图中的运行按钮

步骤 4：查看结果

输出内容	位置	说明
测试结果	调试控制台	显示测试执行结果和通过/失败的测试
错误信息	调试控制台	失败的测试会显示详细的错误信息和堆栈跟踪
变量值	调试视图	在断点处可以查看变量值和调用堆栈

3.2 测试输出

输出类型	位置	说明
控制台输出	VS Code 调试控制台	测试运行过程中的日志信息
测试结果	测试视图	显示通过/失败的测试数量
覆盖率报告	未配置	目前未配置代码覆盖率工具，可在未来添加

4. 测试覆盖范围

4.1 单元测试覆盖

fileCheck 模块

功能	测试项	状态
文件内容解析（ <code>parseFileContent</code> ）	完整注释的函数解析	已覆盖
	多个函数的文件解析	已覆盖
	控制语句过滤（if/for/while）	已覆盖
	注释提取	已覆盖
	位置计算（行号、列号）	已覆盖
	文件类型验证（.c/.cpp）	已覆盖
	空文件处理	已覆盖
	只有注释没有函数的文件	已覆盖
文件检查（ <code>checkFile</code> ）	文件读取和解析	已覆盖
	文件读取错误处理	已覆盖
	不支持的文件类型拒绝	已覆盖
	.cpp 文件支持	已覆盖

functionCheck 模块

功能	测试项	状态
函数注释检查 (<code>checkFunction</code>)	@brief 缺失检测	已覆盖
	@param 缺失检测 (多个参数)	已覆盖
	@return 缺失检测	已覆盖
	void 函数特殊处理 (不需要 @return)	已覆盖
	完整注释验证 (无问题)	已覆盖
	完全没有注释的函数 (多个问题)	已覆盖
	指针参数识别	已覆盖
	问题描述生成	已覆盖

fileWhiteList 模块

功能	测试项	状态
配置读取 (<code>getDocDoctorSettings</code>)	默认配置返回	已覆盖
文件白名单 (<code>isFileWhitelisted</code>)	文件路径前缀匹配	已覆盖
	空白名单处理	已覆盖
函数白名单 (<code>isFunctionWhitelisted</code>)	文件级函数白名单	已覆盖
	全局函数白名单 (使用 "*")	已覆盖
	不在白名单中的函数	已覆盖
返回值类型白名单 (<code>isReturnTypeWhitelisted</code>)	void 类型匹配	已覆盖
	非白名单类型	已覆盖
跳过逻辑 (<code>shouldSkipFunction</code>)	main 函数默认跳过	已覆盖
	启用 <code>checkMainFunction</code> 后不跳过	已覆盖
	白名单文件中的函数跳过	已覆盖

jumpToLocation 模块

功能	状态
文件跳转功能	已覆盖
路径处理（绝对/相对）	已覆盖
错误处理	已覆盖

4.2 集成测试覆盖

projectCheck 模块

功能	测试项	状态
项目全量检查（ <code>checkAllFiles</code> ）	工作区所有文件检查	已覆盖
	问题发现和统计	已覆盖
	文件数量统计	已覆盖
	取消机制	已覆盖
	语法错误文件处理	已覆盖

database 模块

功能	测试项	状态
问题存储（ <code>saveProblemToDB</code> ）	保存问题到数据库	已覆盖
	Mock 模式支持	已覆盖
问题加载（ <code>loadProblemsFromDB</code> ）	从数据库加载问题列表	已覆盖
状态更新（ <code>updateProblemStatusInDB</code> ）	更新问题状态（如标记为已忽略）	已覆盖
清空操作（ <code>clearAllProblems</code> ）	清空所有问题记录	已覆盖
Mock 模式验证	数据库不可用时的降级处理	已覆盖

4.3 扩展测试覆盖

extension 模块

功能	测试项	状态
扩展激活（ <code>activate</code> ）	扩展成功激活	已覆盖
命令注册	doc-doctor 相关命令已注册	已覆盖

4.4 测试覆盖率统计

4.4.1 当前状态

注意：当前项目未配置代码覆盖率工具。测试报告仅提供用例通过/失败统计，缺少代码覆盖率数据。

4.4.2 建议配置覆盖率工具

推荐工具：

- `nyc` (Istanbul)：支持 TypeScript/JavaScript 覆盖率统计，与 VS Code 测试框架兼容
- `c8`：基于 V8 原生覆盖率的替代方案，性能更好

集成步骤建议：

1. 安装覆盖率工具：

```
npm install --save-dev nyc
# 或
npm install --save-dev c8
```

2. 修改 `package.json` 测试脚本：

```
{
  "scripts": {
    "test": "nyc --reporter=text --reporter=html npm run test:run"
  }
}
```

3. 在 `.vscode-test.mjs` 中配置覆盖率收集

4. 设置覆盖率阈值 (`.nycrc.json`)：

```
{
  "check-coverage": true,
  "lines": 80,
  "functions": 80,
  "branches": 75,
  "statements": 80
}
```

覆盖率指标：

- **行覆盖率 (Line Coverage)**：代码行执行百分比
- **分支覆盖率 (Branch Coverage)**：条件分支执行百分比
- **函数覆盖率 (Function Coverage)**：函数调用百分比
- **语句覆盖率 (Statement Coverage)**：语句执行百分比

目标覆盖率：

- 核心业务模块 (`projectCheck.ts`, `functionCheck.ts`, `fileCheck.ts`)：≥ 85%
- 工具模块 (`filewhiteList.ts`, `jumpToLocation.ts`)：≥ 80%
- 数据库模块 (`database.ts`)：≥ 75% (部分依赖 C++ DLL，难以完全测试)

- 整体项目：≥ 80%

CI/CD 集成：

- 在 CI/CD 流水线中生成 HTML 覆盖率报告
- 将覆盖率报告作为构建产物保存
- 覆盖率低于阈值时阻止合并请求

5. 测试用例详细说明

5.1 fileCheck 模块测试用例

测试文件：unit/fileCheck.test.ts

测试套件：parseFileContent

测试用例 1：应该正确解析包含完整注释的函数

项目	内容
测试输入	<code>testFiles.completeComment</code> - 包含完整 Doxygen 注释的函数
输入示例	<pre>c
/**
 * @brief 计算两个整数的和
 * @param a 第一个加数
 * @param b 第二个加数
 * @return 两数之和
 */
int add(int a, int b) {
 return a + b;
}</pre>
预期输出	<code>result.success = true</code> , <code>result.functions.length = 1</code>
断言检查	<code>result.success === true</code> <code>result.functions.length === 1</code> <code>func.functionName === "add"</code> <code>func.functionSignature.includes("int add(int a, int b)")</code> <code>func.comment.includes("@brief")</code> <code>func.comment.includes("@param")</code> <code>func.comment.includes("@return")</code>

测试用例 2：应该正确解析多个函数

项目	内容
测试输入	<code>testFiles.multipleFunctions</code> - 包含多个函数的文件
预期输出	<code>result.success = true</code> , <code>result.functions.length = 3</code>

项目	内容
断言检查	<code>result.functions[0].functionName === "add"</code> <code>result.functions[1].functionName === "subtract"</code> <code>result.functions[2].functionName === "multiply"</code>

测试用例 3: 应该过滤控制语句 (if/for/while)

项目	内容
测试输入	<code>testFiles.controlStatements</code> - 包含控制语句的文件
预期输出	只识别 <code>test</code> 函数, 不识别 <code>if</code> 、 <code>for</code> 、 <code>while</code>
断言检查	<code>functionNames.includes("if") === false</code> <code>functionNames.includes("for") === false</code> <code>functionNames.includes("while") === false</code> <code>functionNames.includes("test") === true</code>

测试用例 4: 应该正确提取函数注释

项目	内容
测试输入	<code>testFiles.completeComment</code>
断言检查	<code>func.comment.includes("/**")</code> <code>func.comment.includes("*/")</code> <code>func.comment.includes("@brief")</code>

测试用例 5: 应该正确计算函数位置 (行号和列号)

项目	内容
测试输入	包含多行注释和函数的代码
断言检查	<code>result.functions[0].lineNumber > 0</code> <code>result.functions[0].columnNumber > 0</code>

测试用例 6: 应该拒绝非 .c/.cpp 文件

项目	内容
测试输入	<code>testFiles.completeComment</code> , 文件名 <code>"test.txt"</code>
预期输出	<code>result.success = false</code> , <code>result.errorCode = "UNSUPPORTED_FILE_TYPE"</code>
断言检查	<code>result.success === false</code> <code>result.errorCode === "UNSUPPORTED_FILE_TYPE"</code>

测试用例 7: 应该处理空文件

项目	内容
测试输入	<code>testFiles.emptyFile</code> - 空字符串
预期输出	<code>result.success = true</code> , <code>result.functions.length = 0</code>
断言检查	1. <code>result.success === true</code> 2. <code>result.functions.length === 0</code>

测试用例 8: 应该处理只有注释没有函数的文件

项目	内容
测试输入	<code>testFiles.onlyComments</code> - 只有注释块, 没有函数定义
预期输出	<code>result.success = true</code> , <code>result.functions.length = 0</code>
断言检查	1. <code>result.success === true</code> 2. <code>result.functions.length === 0</code>

测试套件: `checkFile`

测试用例 1: 应该成功读取并解析文件

项目	内容
测试输入	创建临时文件 <code>test_complete.c</code> , 内容为 <code>testFiles.completeComment</code>
预期输出	<code>result.success = true</code> , <code>result.functions.length = 1</code>
断言检查	1. <code>result.success === true</code> 2. <code>result.functions.length === 1</code> 3. <code>result.functions[0].functionName === "add"</code>

测试用例 2: 应该处理文件读取错误

项目	内容
测试输入	不存在的文件 URI
预期输出	<code>result.success = false</code> , <code>result.errorCode = "READ_ERROR"</code>
断言检查	1. <code>result.success === false</code> 2. <code>result.errorCode === "READ_ERROR"</code>

测试用例 3: 应该拒绝非 `.c/.cpp` 文件

项目	内容
测试输入	创建临时文件 <code>test.txt</code>
预期输出	<code>result.success = false</code> , <code>result.errorCode = "UNSUPPORTED_FILE_TYPE"</code>

项目	内容
断言检查	1. <code>result.success === false</code> 2. <code>result.errorCode === "UNSUPPORTED_FILE_TYPE"</code>

测试用例 4: 应该正确解析 .cpp 文件

项目	内容
测试输入	创建临时文件 <code>test.cpp</code> , 包含 C++ 函数
预期输出	<code>result.success = true</code> , <code>result.functions.length = 1</code>
断言检查	1. <code>result.success === true</code> 2. <code>result.functions.length === 1</code>

5.2 functionCheck 模块测试用例

测试文件: `unit/functionCheck.test.ts`

测试套件: `checkFunction`

测试用例 1: 应该检测缺少 @brief 的问题

项目	内容
测试输入	<code>testFiles.missingBrief</code> - 缺少 @brief 的函数
输入示例	<pre>c
/**
 * @param a 第一个加数
 * @param b 第二个加数
 * @return 两数之和
 */
int add(int a, int b)</pre>
预期输出	<code>problems</code> 数组包含 1 个 <code>BRIEF_MISSING</code> 类型的问题
断言检查	1. <code>briefProblems.length === 1</code> 2. <code>briefProblems[0].problemType === ProblemType.BRIEF_MISSING</code>

测试用例 2: 应该检测缺少 @param 的问题

项目	内容
测试输入	<code>testFiles.missingParam</code> - 缺少 @param 的函数 (有两个参数)
输入示例	<pre>c
/**
 * @brief 计算两个整数的和
 * @return 两数之和
 */
int add(int a, int b)</pre>
预期输出	<code>problems</code> 数组包含 2 个 <code>PARAM_MISSING</code> 类型的问题
断言检查	1. <code>paramProblems.length === 2</code> 2. 所有问题的 <code>problemType === ProblemType.PARAM_MISSING</code>

测试用例 3: 应该检测缺少 @return 的问题

项目	内容
测试输入	<code>testFiles.missingReturn</code> - 缺少 @return 的函数
输入示例	<code>c
/**
 * @brief 计算两个整数的和
 * @param a 第一个加数
 * @param b 第二个加数
 */
int add(int a, int b)</code>
预期输出	<code>problems</code> 数组包含 1 个 <code>RETURN_MISSING</code> 类型的问题
断言检查	<code>returnProblems.length === 1</code> <code>returnProblems[0].problemType === ProblemType.RETURN_MISSING</code>

测试用例 4: void 函数不应该要求 @return

项目	内容
测试输入	<code>testFiles.voidFunction</code> - void 返回类型的函数
输入示例	<code>c
/**
 * @brief 打印消息
 * @param message 要打印的消息
 */
void printMessage(const char* message)</code>
预期输出	<code>problems</code> 数组不包含 <code>RETURN_MISSING</code> 类型的问题
断言检查	<code>returnProblems.length === 0</code>

测试用例 5: 完整注释的函数不应该有问题

项目	内容
测试输入	<code>testFiles.completeComment</code> - 包含完整注释的函数
预期输出	<code>problems</code> 数组为空
断言检查	<code>problems.length === 0</code>

测试用例 6: 完全没有注释的函数应该有多问题

项目	内容
测试输入	<code>testFiles.noComment</code> - 完全没有注释的函数
输入示例	<code>c
int add(int a, int b) {
 return a + b;
}</code>
预期输出	<code>problems</code> 数组包含至少 3 个问题 (BRIEF、2个PARAM、1个RETURN)

项目	内容
断言检查	1. <code>problems.length >= 3</code> 2. <code>problemTypes.includes(ProblemType.BRIEF_MISSING)</code> 3. <code>problemTypes.includes(ProblemType.PARAM_MISSING)</code> 4. <code>problemTypes.includes(ProblemType.RETURN_MISSING)</code>

测试用例 7: 应该正确识别指针参数

项目	内容
测试输入	<code>testFiles.pointerParams</code> - 包含指针参数的函数
输入示例	<pre>c
/**
 * @brief 交换两个整数的值
 * @param a 第一个整数的指针
 * @param b 第二个整数的指针
 */
void swap(int* a, int* b)</pre>
预期输出	如果注释完整, 则没有问题; 如果缺少 @param, 应检测到两个参数
断言检查	1. 参数识别正确 (包括指针类型)

测试用例 8: 问题描述应该包含正确的信息

项目	内容
测试输入	<code>testFiles.missingParam</code>
预期输出	问题描述包含 @param 和参数名 (a 或 b)
断言检查	1. <code>paramProblem.problemDescription.includes("@param")</code> 2. <code>paramProblem.problemDescription.includes("a") paramProblem.problemDescription.includes("b")</code>

5.3 fileWhiteList 模块测试用例

测试文件: `unit/filewhiteList.test.ts`

测试套件: `getDocDoctorSettings`

测试用例	描述	验证点
应该返回默认配置	验证默认配置结构	配置项类型正确

测试套件: `isFileWhitelisted`

测试用例	描述	验证点
应该正确识别文件白名单	验证路径前缀匹配	匹配的文件返回 true

测试用例	描述	验证点
空白名单应该返回 false	验证空配置处理	未匹配的文件返回 false

测试套件：isFunctionWhitelisted

测试用例	描述	验证点
应该正确识别文件级函数白名单	验证文件级白名单匹配	匹配的函数返回 true
应该正确识别全局函数白名单	验证全局白名单 ("*")	全局函数返回 true
不在白名单中的函数应该返回 false	验证未匹配处理	未匹配的函数返回 false

测试套件：isReturnTypeWhitelisted

测试用例	描述	验证点
应该正确识别 void 类型白名单	验证返回值类型匹配	void 函数返回 true
非白名单类型应该返回 false	验证非匹配类型	非 void 函数返回 false

测试套件：shouldSkipFunction

测试用例	描述	验证点
main 函数默认应该跳过	验证 main 函数默认行为	main 函数返回 true
启用 checkMainFunction 后不应该跳过 main	验证配置覆盖	启用后 main 函数返回 false
白名单文件中的函数应该跳过	验证文件白名单优先级	白名单文件中的函数返回 true

5.4 jumpToLocation 模块测试用例

测试文件： `unit/jumpToLocation.test.ts`

注意：具体测试用例需要查看实际文件内容。一般包括：

测试项	说明
文件跳转功能	验证文件跳转功能正常
路径处理	处理绝对路径和相对路径
错误处理	处理文件不存在、函数不存在等错误情况

5.5 projectCheck 模块测试用例

测试文件: `integration/projectCheck.test.ts`

测试套件: ProjectCheck Integration Tests

测试用例 1: 应该检查工作区中的所有文件

项目	内容
测试输入	工作区包含多个测试文件 (file1.c, file2.c, file3.c, file4.c)
预期输出	<code>result.success = true</code> , <code>result.totalFiles > 0</code> , <code>result.checkedFiles > 0</code>
断言检查	<code>result.success === true</code> <code>result.totalFiles > 0</code> <code>result.checkedFiles > 0</code>

测试用例 2: 应该发现注释问题

项目	内容
测试输入	工作区包含有注释问题的文件 (missingBrief, missingParam, noComment)
预期输出	<code>result.problems.length > 0</code>
断言检查	<code>result.success === true</code> <code>result.problems.length > 0</code>

测试用例 3: 应该正确统计检查的文件数

项目	内容
测试输入	工作区包含多个测试文件
预期输出	<code>result.totalFiles >= testFilesList.length</code> , <code>result.checkedFiles <= result.totalFiles</code>
断言检查	<code>result.totalFiles >= testFilesList.length</code> <code>result.checkedFiles <= result.totalFiles</code>

测试用例 4: 应该处理取消操作

项目	内容
测试输入	创建 CancellationTokenSource, 立即调用 <code>cancel()</code>
预期输出	<code>result.success = false</code> , <code>result.errorMessage</code> 包含"取消"
断言检查	<code>result.success === false</code> <code>result.errorMessage?.includes("取消")</code>

测试用例 5: 应该跳过语法错误的文件

项目	内容
测试输入	创建包含语法错误的文件 <code>syntax_error.c</code> (<code>testFiles.syntaxError</code>)
输入示例	<pre>c
int add(int a, int b {
 return a + b;
}</pre>
预期输出	语法错误被记录 (<code>problemType === 5</code>) , 文件被添加到 <code>skippedFiles</code>
断言检查	1. <code>syntaxProblems.length > 0</code> 2. <code>result.skippedFiles.find(f => f.includes("syntax_error.c"))</code> 存在

5.6 database 模块测试用例

测试文件: `integration/database.test.ts`

测试套件: Database Integration Tests

测试用例 1: 应该能够保存问题到数据库

项目	内容
测试输入	创建 <code>ProblemInfo</code> 对象, 包含问题类型、文件路径、函数名等信息
输入示例	<pre>typescript
{
 problemType: ProblemType.BRIEF_MISSING,
 filePath: "test.c",
 functionName: "testFunc",
 functionSignature: "int testFunc()",
 lineNumber: 1,
 columnNumber: 1,
 problemDescription: "缺少函数功能描述",
 functionSnippet: "int testFunc() { return 0; }"
}</pre>
预期输出	<code>result.success = true</code> (Mock 模式下总是返回成功)
断言检查	1. <code>result.success === true</code>

测试用例 2: 应该能够从数据库加载问题

项目	内容
测试输入	调用 <code>loadProblemsFromDB()</code>
预期输出	<code>result.success = true</code> , <code>result.problems</code> 是数组

项目	内容
断言检查	<code>result.success === true</code> <code>Array.isArray(result.problems)</code>

测试用例 3: 应该能够更新问题状态

项目	内容
测试输入	先保存一个问题, 获取 <code>insertedId</code> , 然后更新状态为 <code>ProblemStatus.IGNORED</code>
预期输出	<code>updateSuccess</code> 为布尔值 (Mock 模式下总是返回 true)
断言检查	<code>typeof updateSuccess === "boolean"</code>

测试用例 4: 应该能够清空所有问题

项目	内容
测试输入	调用 <code>clearAllProblems()</code>
预期输出	返回布尔值
断言检查	<code>typeof result === "boolean"</code>

测试用例 5: 应该在数据库不可用时使用 mock 模式

项目	内容
测试输入	C++ DLL 不可用时的数据库操作
预期输出	自动使用 Mock 模式, 所有操作返回成功
断言检查	<code>result.success === true</code> (Mock 模式下总是返回成功)

5.7 extension 模块测试用例

测试文件: `extension.test.ts`

测试套件: Extension Test Suite

测试用例	描述	验证点
Extension should activate	验证扩展激活	激活函数执行成功
Extension commands should be registered	验证命令注册	doc-doctor 命令已注册

6. 测试数据

6.1 testFiles.ts 说明

测试数据文件位于 `fixtures/testFiles.ts`，包含各种 C/C++ 代码样本，用于模拟不同的注释场景。

6.2 测试文件类型

测试文件变量	描述	用途
<code>completeComment</code>	包含完整注释的函数 (@brief、@param、@return)	验证完整注释通过检查
<code>missingBrief</code>	缺少 @brief 的函数	测试 BRIEF_MISSING 检测
<code>missingParam</code>	缺少 @param 的函数	测试 PARAM_MISSING 检测
<code>missingReturn</code>	缺少 @return 的函数	测试 RETURN_MISSING 检测
<code>noComment</code>	完全没有注释的函数	测试多问题检测
<code>voidFunction</code>	void 函数 (不需要 @return)	测试 void 函数特殊处理
<code>mainFunction</code>	main 函数	测试 main 函数跳过逻辑
<code>multipleFunctions</code>	包含多个函数的文件	测试多函数解析
<code>pointerParams</code>	包含指针参数的函数	测试指针参数识别
<code>syntaxError</code>	包含语法错误的文件	测试语法错误处理
<code>emptyFile</code>	空文件	测试空文件处理
<code>onlyComments</code>	只有注释没有函数的文件	测试无函数文件处理
<code>controlStatements</code>	包含控制语句的文件	测试控制语句过滤

6.3 添加新的测试数据

在 `fixtures/testFiles.ts` 中添加新的测试样本：

```
export const testFiles = {
  // ... 现有测试数据

  /**
   * 新的测试场景描述
   */
  newTestScenario: `
// 测试代码内容
`,
};
```

7. 测试工具

7.1 testHelpers.ts 功能说明

测试辅助工具模块提供以下功能：

函数名	签名	功能	参数	返回值	用途
createMockWorkspaceFolder	<code>createMockWorkspaceFolder(name?: string): vscode.WorkspaceFolder</code>	创建模拟的工作区文件夹	<code>name</code> - 工作区名称 (可选, 默认 "test-workspace")	VS Code WorkspaceFolder 对象	为需要工作区上下文的测试提供 Mock 对象
createTestFile	<code>createTestFile(fileName: string, content: string, workspaceFolder?: vscode.WorkspaceFolder): Promise<vscode.Uri></code>	创建测试用的临时文件	<code>fileName</code> - 文件名; <code>content</code> - 文件内容; <code>workspaceFolder</code> - 工作区文件夹 (可选)	文件 URI	在测试中创建临时文件, 测试完成后需要调用 <code>cleanupTestFile</code> 清理
cleanupTestFile	<code>cleanupTestFile(uri: vscode.Uri): void</code>	清理测试文件	<code>uri</code> - 要清理的文件 URI	void	在测试完成后删除临时文件
createMockConfiguration	<code>createMockConfiguration(config?: Record<string, any>): vscode.WorkspaceConfiguration</code>	创建模拟的 VS Code 配置	<code>config</code> - 配置覆盖 (可选)	VS Code WorkspaceConfiguration 对象	为需要配置的测试提供 Mock 配置对象
sleep	<code>sleep(ms: number): Promise<void></code>	等待指定时间 (用于异步测试)	<code>ms</code> - 等待毫秒数	Promise	在需要时间延迟的测试中使用

7.2 Mock 对象说明

VS Code API Mock

测试中使用 Mock 对象模拟 VS Code API，避免依赖真实的 VS Code 环境：

Mock 对象	说明
WorkspaceFolder	模拟工作区文件夹
WorkspaceConfiguration	模拟配置对象
Uri	使用真实的 Uri 对象 (VS Code 提供)

数据库 Mock

当 C++ DLL 不可用时，数据库模块会自动使用 Mock 模式：

特性	说明
操作结果	所有数据库操作返回成功
数据持久化	数据不实际持久化
环境要求	确保测试可以在没有原生模块的环境中运行

附录

相关文档

文档名称	路径	说明
README	README.md	快速参考文档
需求文档	需求文档.md	功能需求说明
设计文档	设计文档.md	系统设计说明